Java Development Tools

JDK

JDK (Java Development Kit) is the set of tools for building Java programs.
It includes the compiler, runtime, and libraries.

IntelliJ IDEA

IntelliJ IDEA is an IDE (integrated development environment) for writing and managing code.
It is especially popular for Java development.

JAVA BASICS

Anatomy of a Java Program

Example

```java
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello!");
    }
}
```

keyword · class/type · method · string

Main parts

Class: HelloWorld contains the program.

Main method: main is where the program starts.

Statement: System.out.println("Hello!") tells Java what to do. Statements end with a semicolon.

Braces: { } mark the beginning and end of code blocks.

Naming conventions

Classes use PascalCase: HelloWorld.

Methods use camelCase and usually start with a verb: printMessage().

A public class name should match its filename: HelloWorld.java.

Packages

A package groups related classes and helps prevent naming conflicts.

Example: package com.example.app; | Folder: com/example/app

Create a Package in IntelliJ IDEA

Packages keep related classes organized and names distinct.

Create the package

1. In the Project panel, expand your project, then src.
2. Right-click src and choose New > Package.
3. Enter a lowercase name, such as

```
com.example.app
```

4. Move Main.java into the package. Choose Refactor if IntelliJ asks.

What IntelliJ adds to Main.java

```
package com.example.app;
```

File path: src/com/example/app/Main.java

The main method

Why the traditional entry point is static

NOTE:

`static` lets Java run **`main`** without creating an object first. Keep it in your program's **`main`** method.

How Java Code Gets Executed

Java code moves through compilation and execution.

1. Compilation: source code to bytecode



Java source code is saved in a **`.java`** file, such as **`Main.java`**. Compile it from the folder containing the file:

```
javac Main.java
```

`javac` is the Java compiler. It creates **`Main.class`**, which contains bytecode - instructions for the Java Virtual Machine. The compiler comes with the JDK, which provides tools for developing Java programs.

2. Execution: bytecode to a running program

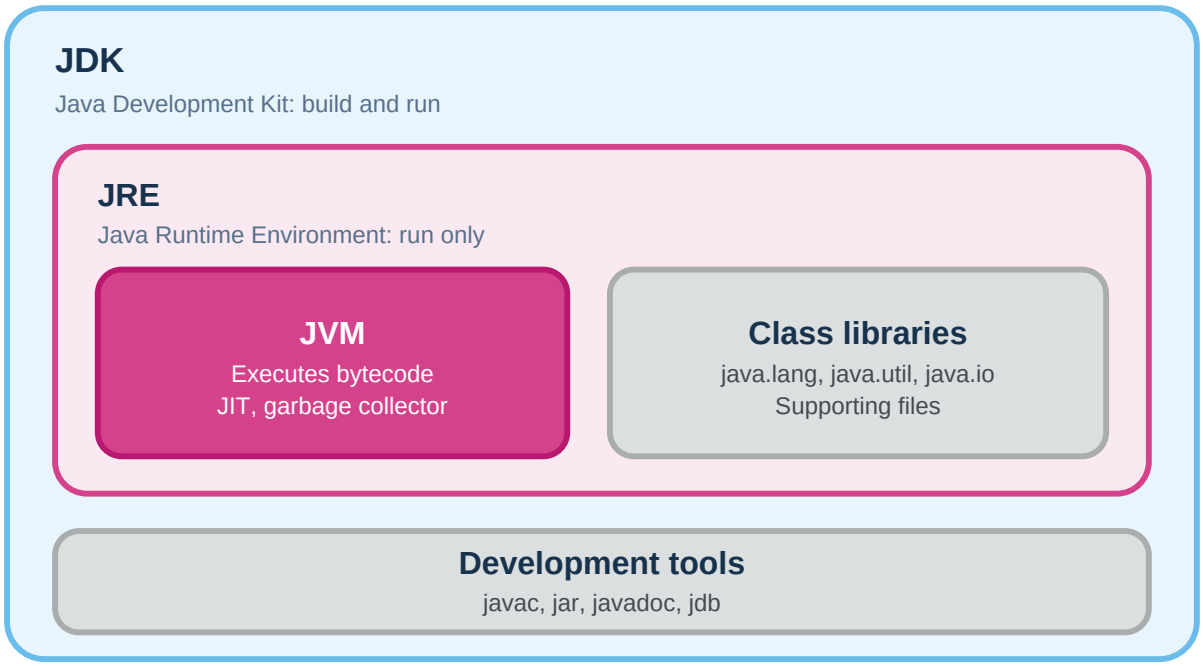


The **JVM (Java Virtual Machine)** loads and runs the bytecode in a `.class` file. It can translate frequently used parts into native machine instructions for the computer.

The **JRE (Java Runtime Environment)** provides the JVM and the Java libraries needed to run programs. In simple terms, the JVM runs the bytecode, and the JRE provides the runtime environment it needs.

Each operating system has a compatible JVM. That lets the same Java bytecode run on different systems without recompiling the source for each one.

3. JDK, JRE and JVM together



Because the JDK is built in layers, installing the outer one installs everything inside it.

- The JDK contains the JRE, since you need a way to run your code after you compile it.
- The JRE contains the JVM, since running Java code means executing bytecode, which only the JVM does.

So one JDK install gives you the compiler and tools (`javac`, `jar`, and so on), the class libraries, and the JVM. When you run `javac` and then `java`, the first uses the JDK's tools and the second launches the JVM from the bundled runtime.

Java Editions



Standard Edition (SE)

General-purpose Java for desktop and server applications.

Enterprise Edition (EE)

Adds APIs and services for large web and business applications.

Micro Edition (ME)

Designed for embedded and resource-limited devices, such as sensors.

Java Card

For secure applications on smart cards and other devices with very limited resources.

For most beginners: start with Java SE. Java EE is now known as Jakarta EE.

Variables

Variables store values that a program can use and update.

```
int age = 30;
System.out.println("Age: " + age);
```

Declare and use a variable

int is the type and stores whole numbers. age is the variable name; 30 is its starting value. The print line joins the label and value, so it displays Age: 30.

```
int temperature = 45;
temperature = 60;
System.out.println("Temperature: " + temperature);
```

Change its value

temperature starts at 45, then is updated to 60. Since it was already declared, the second line uses the name only. The print statement displays Temperature: 60.

```
int myAge = 26;
int herAge = myAge;
System.out.println("Her Age: " + herAge);
```

Copy a value

herAge receives the current value of myAge (26). Each int variable stores its own value, so changing one later does not automatically change the other.

Remember: declare a variable with its type once, then use its name to read or change it.

Primitive Types

Main.java demonstrates five of Java's eight built-in primitive types.

Primitive	Reference / non-primitive
Stores a simple value, such as a number or true/false.	Stores a reference to an object, such as a String or array.

```
byte age = 30;

long viewsCount = 3_123_456_789L;

float price = 10.99F;

char letter = 'A';

boolean isEligible = false;
```

byte small whole number

long large whole number; L marks long, underscores group digits

float decimal value; F marks float

char one character in single quotes

boolean true or false

All eight primitive types

Type	Size	Used for
byte	1 byte	Small whole numbers: -128 to 127
short	2 bytes	Whole numbers: -32,768 to 32,767
int	4 bytes	Whole numbers: -2,147,483,648 to 2,147,483,647
long	8 bytes	Very large whole numbers
float	4 bytes	Decimal values, about 6-7 significant digits
double	8 bytes	Decimal values, about 15-16 significant digits
char	2 bytes	One UTF-16 code unit
boolean	not specified	true or false; Java sets no fixed storage size

Reference (Non-Primitive) Types

Main.java uses Date; String is another common reference type.

Reference type

A reference variable stores a reference to an object. Date and String are examples.

Date example

```
import java.util.Date;


Date now = new Date();

System.out.println(now);
```

What happens

Date is a class in the `java.util` package, so the import lets this file use the short name **Date**. `new Date()` creates an object for the current date and time. **now** refers to that object, and printing it displays its date and time.

String example

```
String message = "Hello, Java!";

System.out.println(message);
```

What happens

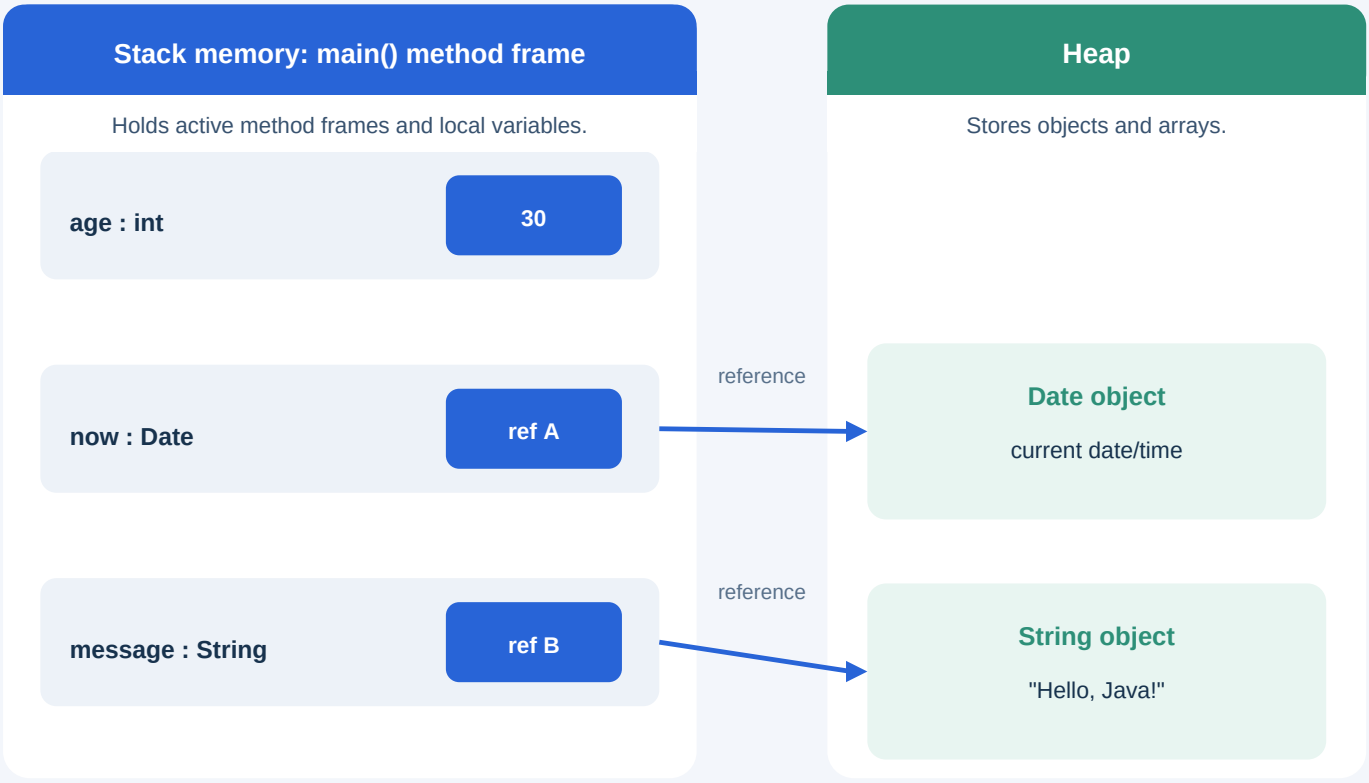
String is also a reference type. The text in double quotes creates the string value, and **message** refers to it. Java makes `String` available automatically, so no import is needed.

Output: Hello, Java!

Important: Java creates a **String** object for a string literal, so `"Hello, Java!"` needs no `new`. **Date** has no literal syntax, so `new Date()` creates the object.

Where Primitive and Reference Variables Live

A simple memory model for local variables inside main().



Primitive local

```
int age = 30;
```

The local variable contains the value **30** directly in the current method frame.

Stack memory: Each thread's stack holds its active method frames and local variables.

Reference local

```
Date now = new Date();
String message = "Hello, Java!";
```

now and message hold references in the stack frame. Their Date and String objects are stored in the heap.

Heap memory: Shared memory where objects and arrays are created.

Remember: This is the JVM's conceptual model for local variables. Runtime optimizations can change their physical placement.

Working with Strings

Useful methods for checking and changing text

Important: String is immutable: these methods return a result without changing the original text.

The examples below all use the same String variable.

Example

```
String text = "  Java Programming  ";
System.out.println(text.length());
System.out.println(text.trim());
System.out.println(text.replace("Java", "Python"));
System.out.println(text.indexOf("Programming"));
System.out.println(text.endsWith(" "));
System.out.println(text.toUpperCase());
System.out.println(text.toLowerCase());
```

What each method does

Method	Purpose	Result for this example
`length()`	Returns the String's length.	`20`
`trim()`	Removes spaces from the beginning and end.	`"Java Programming"`
`replace("Java", "Python")`	Replaces matching text with other text.	`" Python Programming "`
`indexOf("Programming")`	Returns the first match's zero-based position; -1 if absent.	`7`
`endsWith(" ")`	Checks whether the text ends with the given suffix.	`true`
`toUpperCase()`	Returns the text in uppercase.	`" JAVA PROGRAMMING "`
`toLowerCase()`	Returns the text in lowercase.	`" java programming "`

Remember: trim() removes only outer spaces; indexOf() starts counting at 0. endsWith() is case-sensitive. The original text variable stays unchanged.

Escape Sequences

Use a backslash to include special characters in a String

Key idea: A backslash starts an escape sequence inside a Java string literal.

Write the sequence in your code; Java converts it when the program runs.

Examples from Main.java

```
String message = "Hello \"Salik\"";
System.out.println(message);

String directory = "c:\\Windows\\....";
System.out.println(directory);

String tab = "c:\tWindows\\....";
System.out.println(tab);

String newLine = "c:\nWindows\\....";
System.out.println(newLine);
```

What each sequence does

Sequence	Purpose	What the example prints
`\"`	Adds a double quote inside the text.	`Hello "Salik"`
`\\`	Adds one literal backslash.	`c:\Windows\....`
`\t`	Adds a tab character.	`c: [tab] Windows\....`
`\n`	Starts a new line.	`c: then a new line: Windows\....`

Windows paths: write each backslash as \\, because a single backslash begins an escape sequence.

Use "c:\\Windows" in Java; it prints c:\Windows.

Arrays

Store and work with several values of the same type

Array: A fixed-size collection of values, all of the same type.

An array variable refers to an array object; indexes start at 0.

Create and read

Import java.util.Arrays to print every value together.

```
import java.util.Arrays;
int[] numbers = {10, 20, 30};
System.out.println(numbers[0]); // 10
System.out.println(Arrays.toString(numbers));
```

Full-array output: [10, 20, 30].

Update and check size

Assign a new value or read the array's length.

```
numbers[1] = 25;
System.out.println(numbers[1]); // 25
System.out.println(numbers.length); // 3
```

Use .length without parentheses.

Loop through an array

Print each element in order.

```
for (int i = 0; i < numbers.length; i++) {
    System.out.println(numbers[i]);
}
```

Output

```
10
20
30
```

Create an array with a size

Choose the number of elements when creating it.

```
int[] scores = new int[3];
scores[0] = 90;
// {90, 0, 0}
```

Unassigned int elements start at 0.

Remember: Arrays have a fixed size and are reference types.

The last valid index is length - 1. An invalid index causes an error.

Multidimensional Arrays

Organize values in rows and columns

2D array: An array of arrays. Use [row][column] to select a value.

Row and column indexes both start at 0.

Create and access

Access an item by its row and column.

```
int[][] scores = {
    {80, 90, 85},
    {70, 75, 95}
};
System.out.println(scores[0][1]); // 90
```

scores[0][1] means row 0, column 1: 90.

Print the whole array

Use deepToString to show nested values.

```
import java.util.Arrays;
System.out.println(Arrays.deepToString(scores));
```

Output

```
[[80, 90, 85], [70, 75, 95]]
```

Arrays.toString() won't show nested contents.

Loop through every value

The outer loop visits rows; the inner loop visits each row's values.

```
for (int row = 0; row < scores.length; row++) {
    for (int col = 0; col < scores[row].length; col++) {
        System.out.println(scores[row][col]);
    }
}
```

Output

```
80
90
85
70
75
95
```

Remember: scores.length is the number of rows.

scores[row].length is the number of columns in that row.

Constants

A value you assign once and keep unchanged

In Main.java: PI is declared with final, so its value cannot be reassigned.
The program prints the constant's value: 3.14.

Example

```
final float PI = 3.14F;
System.out.println(PI);
```

Output

```
3.14
```

What final means

final lets you assign the variable once.
Trying to change it later causes a compile error:

```
PI = 3.14159F;
```

Because PI is final, Java won't allow this assignment.

The value 3.14 is an approximation of pi.

Naming and float values

Constant names are commonly written in uppercase, such as PI or MAX_USERS.

Decimal literals are double by default. The F suffix marks 3.14F as a float value.

Uppercase is a naming convention; final enforces the rule.

Remember: final controls whether the value can be reassigned.

The uppercase name is a convention that makes constants easy to recognize.

Arithmetic Operators

Calculate values and change a number by one

Operators: + add - subtract * multiply / divide % remainder

The code examples below show each operator in use.

Basic calculations

Use these operators to work with numbers.

```
int a = 10;
int b = 3;
System.out.println(a + b); // 13
System.out.println(a - b); // 7
System.out.println(a * b); // 30
System.out.println(a % b); // 1
```

% gives the remainder after division.

Integer and decimal division

With integers, division drops the fractional part.

```
int whole = 10 / 3; // 3
double decimal = 10.0 / 3;
System.out.println(whole); // 3
System.out.println(decimal); // 3.3333333333333335
```

10.0 is a double, so the result keeps a fractional part.

Increment and decrement

++ adds 1; -- subtracts 1.

```
int count = 5;
count++;
System.out.println(count); // 6
count--;
System.out.println(count); // 5
```

On its own, count++ and ++count both add one.

Order of operations

Multiplication happens before addition; parentheses go first.

```
int result = 2 + 3 * 4; // 14
int grouped = (2 + 3) * 4; // 20
```

Use parentheses when you want to control the order.

Remember: 10 / 3 is integer division and gives 3; 10.0 / 3 gives a decimal result.

Use at least one double value when you want the fractional part of a division.

Casting and Type Conversion

Convert numeric values, or parse text into a number

Main.java: shows automatic conversion, an explicit cast, and parsing text.

Parsing a String to a number is different from casting a numeric value.

Automatic conversion

Java promotes numeric values when an expression needs a wider type.

```
short x = 1;
int y = x + 2;
System.out.println(y); // 3
```

The result is an int, so it fits in y.

Automatic conversion to double

The int value 2 is converted to double during the calculation.

```
double a = 1.1;
double b = a + 2;
System.out.println(b); // 3.1
```

No cast is needed for this widening conversion.

Explicit cast to int

Write the target type in parentheses before the value.

```
double q = 1.1;
int w = (int) q + 2;
System.out.println(w); // 3
```

(int) q drops the .1 first; casting truncates, it does not round.

Parse text into a number

Use Integer.parseInt when the value is stored as text.

```
String s = "1";
int r = Integer.parseInt(s) + 2;
System.out.println(r); // 3
```

This is parsing, not casting: a String is not a numeric type.

Remember: Widening is often automatic; narrowing needs an explicit cast.

Casting can lose information: (int) 1.9 becomes 1, not 2.

The Math Class

Useful built-in methods for calculations and random numbers

Math provides ready-made static methods.
Call them with Math.methodName(...); no import or Math object is needed.

Round, ceil, and floor

These methods change a decimal value in different ways.

```
Math.round(1.1F);
(int) Math.ceil(1.1F);
(int) Math.floor(1.1F);
```

Output 1; 2; 1

ceil rounds upward; floor rounds downward. Both return double, so the cast makes an int.

Smallest and largest

Compare two values with min and max.

```
Math.min(1, 2);
Math.max(1, 2);
```

Output 1; 2

min returns the smaller value; max returns the larger one.

A random decimal

Math.random() gives a new pseudo-random decimal each time.

```
double result = Math.random();
System.out.println(result);
```

Output For example: 0.583...

The value is at least 0.0 and always less than 1.0.

A random whole number

Multiply by 100, then cast to int to remove the decimal part.

```
int randomInt =
    (int) (Math.random() * 100);
System.out.println(randomInt);
```

Output For example: 42

randomInt can be any whole number from 0 through 99.

Remember: Math methods are static, so call them using the class name: Math.methodName(...).

The random integer example uses randomInt instead of the placeholder name x.

Number Formatting

Format numbers as currency or percentages

What is NumberFormat?

NumberFormat is a Java class that turns numbers into readable, formatted text.

It provides formatter objects for styles such as currency and percentages.

Currency

NumberFormat is the variable type. The factory method creates a formatter for money.

```
NumberFormat currency =
    NumberFormat.getCurrencyInstance();
String currencyResult =
    currency.format(1234567.891);
System.out.println(currencyResult);
```

Example output

Example: $1,234,567.89 (varies by locale).

currency is a formatter object. Its format() method returns a String with currency symbols and separators for the active locale.

Percentage

This factory method creates a formatter for percentage values.

```
NumberFormat percentage =
    NumberFormat.getPercentInstance();
String percentageResult =
    percentage.format(0.1);
System.out.println(percentageResult);
```

Example output

Example: 10% (format may vary by locale).

percentage is a formatter object. format() scales 0.1 by 100 and adds the locale's percent symbol.

Important: Why no new keyword?

NumberFormat is an abstract class, so you cannot create it directly with new.

Instead, getCurrencyInstance() creates and returns a suitable formatter for your locale:

```
NumberFormat currency =   NumberFormat.getCurrencyInstance();
```

An object is still created; the factory method creates it and gives it back to you.

JAVA BASICS

Reading Input

Use Scanner to read values typed by the user

First, import Scanner and create it inside main():

```
import java.util.Scanner;   Scanner scanner = new Scanner(System.in);
```

nextLine()

Reads the rest of the current line, including spaces.

```
String fullName = scanner.nextLine();
```

Input: Ada Lovelace -> fullName stores both words.

next()

Reads one token, stopping at whitespace.

```
String firstWord = scanner.next();
```

Input: Ada Lovelace -> firstWord is "Ada".

nextByte()

Reads a whole number that fits in byte range (-128 to 127).

```
byte age = scanner.nextByte();
```

Example input: 25 -> age stores 25.

nextFloat()

Reads a decimal number as a float.

```
float price = scanner.nextFloat();
```

Example input: 12.5 -> price stores 12.5f.

nextInt()

Reads a whole number as an int.

```
int quantity = scanner.nextInt();
```

Example input: 7 -> quantity stores 7.

There is no nextString()

Scanner has no nextString() method. Use one of these:

```
String word = scanner.next();
String line = scanner.nextLine();
```

Use next() for one word, or nextLine() for a full line.

Why can nextLine() return an empty line after a number?

nextInt() reads the number but leaves the Enter key in the input.

The nextLine() may read that Enter and return an empty line.

Clear the leftover Enter, then call nextLine() again to read the text:

```
int age = scanner.nextInt();
scanner.nextLine();  // consume Enter
String name = scanner.nextLine();
```